Table des matières

Cours 05 — Les codes 1
Étape 1 — Les tests génériques · MARTS 1
Étape 2 — Le grain et le témoin, en tests singuliers · MARTS 3
Étape 3 — L'histoire des prix, `snp_prix` · STAGING 4
Étape 4 — Le prix d'époque · INTERMEDIATE 5

Cours 05 — Les codes

Cours 5/9 · La Brûlerie — Formation Analytics Engineering

STAGING	INTERMEDIATE	MARTS
reprendre la `RAW`, une vue par table	joindre, clarifier, préparer l'étoile	concevoir et livrer les étoiles
✓ ici · Fiabiliser et faire tourner, ce qui traverse les trois couches : les tests, l'incrémental, l'intégration continue	✓	✓

À ouvrir après avoir essayé. Le code garde les commentaires numérotés du cours, pour comparer ligne par ligne. Chaque fichier est rangé sous son chemin, et chaque requête du notebook sous son étape ; les requêtes se tapent dans le notebook de DuckDB (`duckdb -ui`, lancé depuis `Projet_Brulerie`), une par cellule, sous celle qui attache l'entrepôt.

Étape 1 — Les tests génériques · MARTS

02_ae/models/marts/_marts.yml

```yaml
# 02_ae/models/marts/_marts.yml : l'étoile des ventes, décrite et testée
models:
  # 1. les dimensions : une clé unique et toujours remplie
  - name: dim_date
    description: "Les 30 jours de juin 2025, un par ligne, fabriqués sans source."
    columns:
      - name: date_jour
        description: "La clé : la date elle-même."
        data_tests:
```


© 2026 iAlexMG. Tous droits réservés.

```yaml
        - unique
        - not_null
- name: dim_produit
  description: "Un produit du catalogue par ligne, au 2025-06-30."
  columns:
    - name: cle_produit
      description: "La clé de substitution, md5 du numéro du produit."
      data_tests:
        - unique
        - not_null
    # 2. une liste fermée, et son not_null à côté
    - name: categorie
      data_tests:
        - not_null
        - accepted_values:
            arguments:
              values: ['ACCESSOIRE', 'BOISSON_CHAUDE', 'BOISSON_FROIDE', 'GRAINS', 'SANDWICH', 'VIENNOISERIE']
- name: dim_membre
  description: "Un membre du programme de fidélité par ligne, sans rien de personnel."
  columns:
    - name: cle_membre
      description: "La clé de substitution, md5 du numéro de la carte."
      data_tests:
        - unique
        - not_null
# 3. le fait : chaque clé recalculée existe dans sa dimension
- name: fct_ventes
  description: "Le fait des ventes, une ligne par ligne d'un ticket payé."
  columns:
    - name: cle_date
      data_tests:
        - not_null
        - relationships:
            arguments:
              to: ref('dim_date')
              field: date_jour
    - name: cle_produit
      data_tests:
        - not_null
        - relationships:
            arguments:
              to: ref('dim_produit')
```


© 2026 iAlexMG. Tous droits réservés.

```
              field: cle_produit
      - name: cle_membre
        description: "Vide pour un ticket sans carte de membre."
        data_tests:
          - relationships:
              arguments:
                to: ref('dim_membre')
                field: cle_membre
```

Étape 2 — Le grain et le témoin, en tests singuliers · MARTS

02_ae/tests/assert_grain_fct_ventes.sql

```sql
-- 02_ae/tests/assert_grain_fct_ventes.sql : aucun couple ticket et ligne deux fois dans le fait
-- 1. les couples (ticket, ligne) que le fait porte plus d'une fois : chaque ligne rendue est un échec
select ticket_id, ligne, count(*) as fois
from {{ ref('fct_ventes') }}
group by ticket_id, ligne
having count(*) > 1
```

02_ae/tests/assert_temoin_caisse.sql

```sql
-- 02_ae/tests/assert_temoin_caisse.sql : chaque ticket payé, au cent près, dans le fait et à la caisse
with fait as (
    -- 1. le montant de chaque ticket, additionné depuis les lignes du fait
    select ticket_id, sum(montant) as montant
    from {{ ref('fct_ventes') }}
    group by ticket_id
),
caisse as (
    -- 2. le sous-total de chaque ticket payé, tel que la caisse l'a imprimé
    select ticket_id, sous_total
    from {{ ref('stg_tickets') }}
    where statut = 'PAYE'
)
-- 3. un ticket d'un seul côté, ou deux montants qui diffèrent : chaque ligne rendue est un échec
select coalesce(f.ticket_id, c.ticket_id) as ticket_id, f.montant, c.sous_total
from fait as f
full outer join caisse as c on c.ticket_id = f.ticket_id
where f.ticket_id is null or c.ticket_id is null or f.montant <> c.sous_total
```


© 2026 iAlexMG. Tous droits réservés.

Étape 3 — L'histoire des prix, `snp_prix` · STAGING

02_ae/models/staging/_sources.yml

```yaml
# 02_ae/models/staging/_sources.yml : la RAW reçue, déclarée une seule fois
sources:
  - name: brulerie
    description: "La RAW du café, tout en texte : chaque export gardé, même deux fois."
    # 1. les tables de la RAW vivent dans le schéma par défaut de DuckDB
    schema: main
    # 2. une entrée par table qu'un modèle lit
    tables:
      - name: raw_tickets
        description: "Les tickets de la caisse, un par ligne et par export."
      - name: raw_lignes
        description: "Les lignes des tickets, un produit vendu par ligne et par export."
      - name: raw_produits
        description: "Le catalogue de la caisse au 2025-06-30."
      - name: raw_membres
        description: "Les membres du programme de fidélité au 2025-06-30."
      - name: raw_modifications
        description: "Le journal des modifications du catalogue, une ligne par valeur changée, depuis 2022."
```

02_ae/snapshots/snp_prix.sql

```sql
{# 02_ae/snapshots/snp_prix.sql : le prix de chaque produit, une version par prix, tirée du journal #}
{% snapshot snp_prix %}
{{ config(
    unique_key='produit_id', strategy='check', check_cols=['prix'], updated_at='en_vigueur_le'
) }}
select
    {# 1. le produit, son prix, et le jour où ce prix est entré en vigueur #}
    produit_id::integer as produit_id,
    nouvelle_valeur::decimal(10, 2) as prix,
    date_trunc('day', horodatage::timestamp) as en_vigueur_le
from {{ source('brulerie', 'raw_modifications') }}
{# 2. le catalogue au jour regardé : les créations et les changements de prix connus ce jour-là #}
where champ in ('CREATION', 'PRIX')
    and horodatage::date <= '{{ var("jour", "2025-06-30") }}'
{# 3. de chaque produit, le dernier prix connu #}
qualify row_number() over (partition by produit_id order by horodatage desc) = 1
{% endsnapshot %}
```


© 2026 iAlexMG. Tous droits réservés.

3.7 Le prix du Latte au 15 juin 2024

```sql
-- 1. le prix du Latte en vigueur le 15 juin 2024
select prix
from brulerie.snp_prix
where produit_id = 105
  and dbt_valid_from <= date '2024-06-15'
  and (dbt_valid_to is null or date '2024-06-15' < dbt_valid_to);
```

Étape 4 — Le prix d'époque · INTERMEDIATE

02_ae/tests/assert_prix_d_epoque.sql

```sql
-- 02_ae/tests/assert_prix_d_epoque.sql : chaque ligne vendue au prix en vigueur le jour de la vente
select v.ticket_id, v.ligne, v.date_vente, v.prix_unitaire, p.prix
-- 1. chaque ligne vendue, avec la version du prix de son produit en vigueur ce jour-là
from {{ ref('int_ventes') }} as v
left join {{ ref('snp_prix') }} as p
    on p.produit_id = v.produit_id
    -- 2. ouverte au plus tard le jour de la vente, et pas encore fermée ce jour-là
    and p.dbt_valid_from <= v.date_vente
    and (p.dbt_valid_to is null or v.date_vente < p.dbt_valid_to)
-- 3. une ligne sans version, ou vendue à un autre prix : chaque ligne rendue est un échec
where p.prix is null or v.prix_unitaire <> p.prix
```


© 2026 iAlexMG. Tous droits réservés.